

Reviewer Pack — Noncommutative L^p Geometry Invariant Bounds for DISJOINTNES...

Entry aa45f3d9592d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 14:51:49 UTC

Noncommutative L^p Geometry Invariant Bounds for DISJOINTNESS Communication Complexity

Entry ID: aa45f3d9592d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 14:51:49 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative L^p Geometry **Field B** (complexity object): Communication Complexity: DISJOINTNESS

Statement:

[‘There exists a constant $c > 0$ such that the L^p spectral radius of any noncommutative L^p space associated with an n -bit Boolean function F , where $p \in (1, \infty)$, is bounded below by $c * \log(n)$ when F is a disjunctive Boolean function.’, ‘For any given disjunctive Boolean function F , the quantity $L^p(\rho(F)) \leq c * \log(n)$, where $\rho(F)$ is the spectral radius of the noncommutative L^p space corresponding to F .’, ‘This bound holds for all $p \in (1, \infty)$.’]

Rationale (proposer’s reasoning):

[‘Noncommutative L^p geometry has been explored in the context of quantum information and may provide new insights into the communication complexity of problems like DISJOINTNESS.’, ‘The spectral radius as an invariant captures geometric properties that could potentially relate to the complexity measures of Boolean functions, particularly those with disjunctive structures.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9175255ee7f9c5e8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all disjunctive Boolean functions F with n bits, the average L^p spectral radius across multiple seeds ($L^p(\rho(F))$) is within a factor of $c * \log(n) \pm 3$ standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - (Noncommutative L^p Geometry) AND (Communication Complexity AND DISJOINTNESS) - (Spectral radius OR L^p spectral radius) AND (Boolean function AND disjunctive) - (Invariant bound) AND (L^p geometry) AND (communication complexity)

Top relevant hits considered: - [http://arxiv.org/abs/1501.00671v1] On the Complexity of Non-commutative Polynomial Factorization - [http://arxiv.org/abs/2006.13712v1] Disjointness through the Lens of Vapnik-Chervonenkis Dimension: Sparsity and Beyond - [http://arxiv.org/abs/math/0412256v2] Trapped submanifolds in Lorentzian geometry - [http://arxiv.org/abs/2601.13953v3] Differentiable Logic Synthesis: Spectral Coefficient Selection via Sinkhorn-Constrained Composition - [http://arxiv.org/abs/0808.0684v1] 9-variable Boolean Functions with Nonlinearity 242 in the Generalized Rotation Class - [http://arxiv.org/abs/2107.14760v1] The spectral form of Koopman representations of the group of measurable functions with values in the circle - [http://arxiv.org/abs/0909.1426v9] L^p boundedness of the Hilbert transform - [http://arxiv.org/abs/1611.00827] Geometric complexity theory and matrix powering

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_disjunctive_boolean_function(n):
        # Generate a disjunctive Boolean function (e.g., OR of all variables)
        return [1 if i % 2 == 0 else 0 for i in range(n)]

    def spectral_radius(matrix):
        n = len(matrix)
        identity = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        max_iter = 1000
        tol = 1e-6

        # Power iteration method to approximate the spectral radius
        v = [1] * n
        for _ in range(max_iter):
```

```

        Av = [sum(matrix[i][j] * v[j] for j in range(n)) for i in range(n)]
        max_val = max(abs(x) for x in Av)
        if max_val < tol:
            break
        v = [x / max_val for x in Av]

    return max_val

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0

for n in n_values:
    F = generate_disjunctive_boolean_function(n)
    matrix = [[F[i] * F[j] for j in range(n)] for i in range(n)]
    radius = spectral_radius(matrix)
    total_metric_value += radius
    instances_tested += 1

mean_metric_value = total_metric_value / len(n_values)
std_deviation = math.sqrt(sum((radius - mean_metric_value) ** 2 for radius in [spectral_radius

conjecture_holds = all(mean_metric_value >= c * math.log(n) for n in n_values)
if not conjecture_holds:
    counterexample = "mean_metric_value < c * log(n)"
else:
    counterexample = ""

return {
    "metric_name": "L^p spectral radius",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fractions}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fractions}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])

```


#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	22693

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 103682 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/aa45f3d9592d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/aa45f3d9592d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/aa45f3d9592d.tar.gz (if generated)